Oracle C2M SSO, SSL & Certificate/Keystore Configuration

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Oracle C2M SSO, SSL & Certificate/Keystore Configuration 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 5

1. Solution Overview — Oracle C2M SSO, SSL & Certificate/Keystore Configuration 5

What this solution is . . . 5

The C2M security model (what fits together) . . . 5

How it works (end to end) . . . 6

What you configure (the levers) . . . 6

Naming convention (used in every doc and script) . . . 6

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 7

2. Requirements Guide — Oracle C2M SSO, SSL & Certificate/Keystore Configuration 7

2.1 WebLogic domain administrative access (Console + WLST) . . . 8

2.2 OS product account, product home, and OUAF tooling . . . 8

2.3 Certificate strategy and Certificate Authority . . . 8

2.4 Server hostnames / SAN plan and fully-qualified URLs . . . 8

2.5 Identity Provider / SSO integration decision . . . 8

2.6 Keystore passphrase custody and secrets handling . . . 9

2.7 Change control, backups, and a rollback path . . . 9

2.8 A non-production pilot server / environment . . . 9

Checklist . . . 9

Global Requirements — Oracle Utilities C2M / WebLogic Solutions 10

1. Licensed Oracle Utilities C2M + WebLogic entitlement . . . 10

2. Certified platform stack (OS, Java, Database, WebLogic) . . . 10

3. Environment topology (Dev / Test / Prod) & access . . . 10

4. Administrative access & tooling (WLST, shell, SQL) . . . 11

5. Change control, backup & a non-production pilot . . . 11

6. Collaborating teams & support model . . . 11

3. Implementation Guide — Oracle C2M SSO, SSL & Certificate/Keystore Configuration 12

Step 0 — Assess the current state (Assess) . . . 12

Step 1 — Create the identity keystore and CSR (Configure) . . . 12

Step 2 — Build the trust keystore and complete the chain (Configure) . . . 13

Step 3 — Point WebLogic at the keystores (Configure) . . . 13

Step 4 — Enable SSL/TLS on the domain (Configure) . . . 13
Step 5 — Align C2M/OUAF to HTTPS (Configure) . . . 14
Step 6 — Configure the SSO Identity Assertion Provider (Configure) . . . 14
Step 7 — Switch OUAF to accept the asserted identity (Configure) . . . 14
Step 8 — Pilot two-way SSL where required (Pilot) . . . 15
Step 9 — Validate the pilot (Validate) . . . 15
Step 10 — Expand across the domain (Expand) . . . 15
5. Testing Guide — Oracle C2M SSO, SSL & Certificate/Keystore Configuration 15
Test 1 — Baseline captured . . . 16
Test 2 — Keystores are valid and chained . . . 16
Test 3 — One-way SSL/TLS terminates correctly . . . 16
Test 4 — Minimum TLS and cipher policy enforced . . . 16
Test 5 — C2M advertises HTTPS . . . 16
Test 6 — SSO sign-in works . . . 16
Test 7 — SSO negative path is safe . . . 16
Test 8 — Two-way SSL enforcement (only where configured) . . . 17
Test 9 — Validation and coverage are clean . . . 17
Test 10 — Domain expansion is consistent . . . 17
Pass/fail gate . . . 17
6. Troubleshooting Guide — Oracle C2M SSO, SSL & Certificate/Keystore Configuration 17
A. Keystore / trust errors . . . 17
B. SSL/TLS won't terminate or is untrusted . . . 18
C. C2M still using HTTP . . . 18
D. SSO does not sign the user in . . . 18
E. Two-way SSL problems . . . 19
F. Certificate expiry / rotation . . . 19
G. Change made no difference / was lost . . . 19
H. Diagnostics quick reference . . . 19
I. When to escalate . . . 19
7. Best Practices — Oracle C2M SSO, SSL & Certificate/Keystore Configuration 20
8. Deliverables Summary — Oracle C2M SSO, SSL & Certificate/Keystore Configuration 21
Documentation . . . 21
Automation scripts (WLST / shell / SQL — never PowerShell) . . . 21
What the buyer receives . . . 22
Outcome . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary — Oracle C2M SSO, SSL & Certificate/Keystore Configuration

Oracle Utilities Customer to Meter (C2M) runs on the Oracle Utilities Application Framework (OUAF) deployed on Oracle WebLogic Server 14c. For the system to be secure and trusted, three things have to be configured correctly and in the right order: **Single Sign-On (SSO)** so users authenticate once through the enterprise Identity Provider, **SSL/TLS** so every connection to the WebLogic domain and to C2M is encrypted, and the **certificate keystores** — the WebLogic identity keystore (the server's private key and certificate) and the trust keystore (the CA certificates it trusts) — that make the TLS handshake and SSO trust actually work. Get any one wrong and you get broken logins, browser warnings, untrusted server-to-server calls, or audit findings.

This solution is a complete, production-safe, deploy-it-on-a-pilot-first package for configuring the C2M security plumbing using only the WebLogic Administration Console, **WLST (Jython)**, `keytool`, and OUAF's own template-driven configuration (`configureEnv` / `initialSetup`) — no third-party tooling. It is written for an experienced C2M/WebLogic administrator and is grounded in Oracle's official WebLogic 14.1.1 security and OUAF Server Administration documentation; version-specific object names are flagged to verify for your release, and My-Oracle-Support-gated notes are pointed to rather than copied.

What you get

- A plain-language Solution Overview of the C2M security model — SSO, SSL/TLS termination, and the identity/trust keystore chain — with a reused naming convention.
- A Requirements Guide with verify / implement-if-missing for every prerequisite.
- An Implementation Guide of exact, ordered steps: build keystores with `keytool`, enable SSL/TLS on the domain via WLST/Console, align C2M to HTTPS through `configureEnv`/`initialSetup`, wire the SSO Identity Assertion Provider, switch OUAF to `CLIENT-CERT`, and (where required) enforce two-way SSL.
- Five automation scripts (WLST/Jython and shell) for read-only discovery, validation, a coverage report, safe remediation (dry-run by default), and a pilot-vs-domain scope helper — none of which ever print private keys or passphrases.
- Testing and Troubleshooting guides plus Best Practices for certificate lifecycle and key rotation.

What this delivers

A measured security baseline of every WebLogic server, correctly built identity and trust keystores, a domain encrypted with a modern TLS floor, C2M advertising HTTPS end to end, SSO sign-in through the enterprise IdP with a safe negative path, and a coverage report proving it — all validated on a pilot before the domain is expanded, with certificate expiry tracked and rotation scheduled.

The deployment model

Assess → Configure → Pilot → Validate → Expand.

Outcome

C2M secured end to end — SSO working, SSL/TLS valid, keystores trusted and managed — proven on a pilot server before you scale, with a documented rollback and no secrets exposed by the tooling.

1. Solution Overview — Oracle C2M SSO, SSL & Certificate/Keystore Configuration

What this solution is

Oracle Utilities Customer to Meter (C2M) runs on the Oracle Utilities Application Framework (OUAF) deployed as JEE web applications on **Oracle WebLogic Server 14c** (14.1.1). Three things have to be configured correctly, in the right order, for the system to be secure and for users and integrations to trust it: **Single Sign-On (SSO)** so people sign in once through the enterprise Identity Provider; **SSL/TLS** so every connection to the WebLogic domain and to C2M is encrypted; and the **certificate keystores** — the WebLogic *identity* keystore (the server's private key and certificate) and the *trust* keystore (the CA certificates the server trusts) — that make the TLS handshake and SSO trust actually work. This solution is a repeatable, production-safe way to configure all three using the WebLogic Administration Console, **WLST (Jython)**, `keytool`, and OUAF's template-driven configuration (`configureEnv` / `initialSetup`) — no third-party tooling.

> ***Version context.*** *Authored for **C2M 25.4 (minimum OUAF v2.9 lineage)** on **WebLogic Server 14.1.1**. WebLogic keystore/SSL steps follow Oracle's *Administering Security for Oracle WebLogic Server 14.1.1* guide; OUAF web-application security follows the *OUAF Server Administration Guide*. Version-specific object names and paths are flagged **"verify for your version"** where they drift between releases. Certain SSO/IdP integration notes and patches are entitlement-gated on **My Oracle Support (MOS)** and are pointed to, not reproduced. See* `global-requirements-oracle.md`.

The C2M security model (what fits together)

1. **Identity keystore** — a keystore (JKS or PKCS12) holding the WebLogic server's **private key** and its signed **server certificate** (plus the CA chain). This is what the server presents during the TLS handshake. Configured per server as **Custom Identity**.
1. **Trust keystore** — a separate keystore holding the **CA certificates** WebLogic trusts (root and intermediate). This is what the server uses to validate certificates presented to it (e.g. a client certificate in two-way SSL, or an IdP/back-end certificate). Configured as **Custom Trust**.
1. **SSL/TLS on the domain** — each WebLogic server has its **SSL listen port** enabled, points at the identity keystore via a **private key alias**, sets **minimum TLS protocol** and cipher policy, and sets **client-certificate enforcement** (one-way vs two-way SSL).
1. **SSL/TLS reaching C2M** — the OUAF web applications are addressed over HTTPS. OUAF's own configuration (the URL/protocol/port it advertises, set through `configureEnv` and materialized by `initialSetup` into `web.xml` / `weblogic.xml` / `spl.properties`) is aligned with the WebLogic SSL endpoint so links, callbacks, and inbound web services all use `https`.

1. **SSO** — user authentication is delegated to the enterprise Identity Provider. The perimeter

(SAML/OIDC or an access-manager agent) authenticates the user and asserts identity to WebLogic; a WebLogic **Identity Assertion Provider** consumes the assertion/header, and the OUAF web application is set to the `CLIENT-CERT` authentication method so it accepts the asserted identity instead of prompting with its own form login.

How it works (end to end)

1. A user hits the C2M URL. The connection is TLS-encrypted using the WebLogic **identity** certificate.
2. The perimeter/IdP authenticates the user (SSO) and asserts identity to WebLogic.
3. WebLogic's **Identity Assertion Provider** validates the assertion (using the **trust** keystore for

the signing certificate where applicable) and establishes the WebLogic subject.

1. OUAF, configured for `CLIENT-CERT`, accepts the asserted user, maps them to a C2M user, and the

session proceeds — no separate C2M password prompt.

1. Server-to-server calls (inbound web services, integrations, batch) use the same TLS endpoints and

the same trust chain, so every hop is encrypted and mutually validated.

What you configure (the levers)

• **Which keystore strategy:** Custom Identity + Custom Trust (recommended) versus demo keystores

(never in production). Separate identity and trust keystores are the standard.

• **Certificate source:** enterprise CA-signed (production) vs self-signed (lower environments only),

and the CA chain imported into trust.

• **SSL posture:** SSL listen port, minimum TLS version (TLS 1.2+), cipher policy, and

client-cert enforcement (`none` / `want` / `need`).

• **SSO integration type:** SAML, OIDC, or an access-manager agent, and the Identity Assertion Provider

that consumes it; plus the OUAF `CLIENT-CERT` auth-method switch.

• **Scope order:** which single server/environment is piloted first before the domain is expanded.

Naming convention (used in every doc and script)

Every artifact this solution creates is named `<Purpose> – <Scope> – <Stage>` and reused verbatim across the docs, scripts, WebLogic console, and `keytool` aliases. The standard set:

Artifact	Name
Identity keystore file	`C2MSEC-Identity-<env>.jks` (e.g. `C2MSEC-Identity-Pilot.jks`)
Trust keystore file	`C2MSEC-Trust-<env>.jks`

Artifact	Name
Server private-key / certificate alias	`c2msec-server`
Root CA alias in trust	`c2msec-rootca`
Intermediate CA alias in trust	`c2msec-intca`
WebLogic Identity Assertion Provider	`C2MSEC-SSO-Asserter`
Discovery export	`Security Baseline – C2M – Discovery`
Validation result	`Security Validation – C2M – Validation`
Coverage/health report	`Security Coverage – C2M – Report`
Pilot scope target	`Security Target – Pilot – Stage1`
Domain scope target	`Security Target – Domain – Stage2`

> *Keystore **passphrases and private keys are never written into docs, scripts, or exports.** Scripts read state and print aliases, file names, ports, expiry, and enforcement — never key material or passwords.*

Limitations (be honest with buyers)

- **SSO depends on an external IdP.** This solution configures the WebLogic/OUAF side (asserter + `CLIENT-CERT`); the IdP/access-manager side (SAML metadata, OIDC client, agent) is owned by the identity team and must be provisioned in parallel.
- **Certificate lifecycle is ongoing.** Certificates expire; a keystore that is correct today fails on the expiry date. This solution includes discovery of expiry and a rotation practice, but rotation must be scheduled, not assumed.
- **Two-way SSL raises the bar.** Requiring client certificates (`need`) protects endpoints but breaks any caller that lacks a trusted client cert — pilot it before enforcing domain-wide.
- **Template overwrite risk.** OUAF regenerates `web.xml` / `weblogic.xml` from templates on `initialSetup`; changes made only to the generated files are lost. Persist changes through `configureEnv`/templates (see Requirements and Implementation).

The standard deployment model

> *Assess the current security state → configure keystores, SSL, then SSO → prove it on a **pilot** server/environment → validate secure sign-in and endpoints → expand across the domain. Pattern: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit gates and a documented rollback (config backup + demo-keystore/previous-keystore fallback) for production changes.*

2. Requirements Guide — Oracle C2M SSO, SSL & Certificate/Keystore Configuration

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (entitlement, certified stack, environments, tooling, change control, teams) are in `global-requirements-oracle.md`.

2.1 WebLogic domain administrative access (Console + WLST)

Description: WebLogic Administration Server reachable, with the admin credentials and **WLST (Jython)** connectivity to the domain that hosts the C2M/OUAF managed servers. **Why:** Keystore, SSL, and Identity Assertion Provider configuration are performed through the Administration Console and WLST against server MBeans. **Verify:** `java weblogic.WLST` then `connect(...)` to the Administration Server succeeds; the Console opens at the admin URL and lists the C2M managed servers. **Implement:** Obtain the WebLogic admin account and the Administration Server URL/port from the infrastructure team; confirm the domain home and `setDomainEnv` are sourced for WLST.

2.2 OS product account, product home, and OUAF tooling

Description: The OUAF/product OS account with access to the C2M **product home** ($SPLEBASE) and the OUAF utilities `configureEnv` and `initialSetup`, plus a JDK on the PATH that provides `keytool`. **Why:** OUAF advertises its own URL/protocol and JEE descriptors (`web.xml`, `weblogic.xml`, `spl.properties`) through `configureEnv`/`initialSetup`; keystores are created and inspected with `keytool` from the JDK. **Verify:** `echo $SPLEBASE` resolves; `configureEnv -h` runs; `keytool -help` runs and `java -version` matches the certified JDK. **Implement:** Log in as the product account, source the environment (`splenviron`/`configureEnv` per your install), and confirm the certified JDK is the one on PATH.

2.3 Certificate strategy and Certificate Authority

Description: A decision on certificate source per environment — **enterprise CA-signed** for production (and ideally test), **self-signed** acceptable only for lower environments — and access to the CA (internal PKI or public CA) to sign a Certificate Signing Request (CSR). **Why:** The identity keystore must hold a certificate that clients and integrations trust; a self-signed or untrusted cert produces browser warnings and failed server-to-server TLS. **Verify:** You can state which CA will sign each environment's certificate and how a CSR is submitted and returned; the CA **root** (and any **intermediate**) certificates are available as files. **Implement:** Engage the PKI/security team to establish the CA path and CSR submission process; obtain the root/intermediate CA certificate files for the trust keystore.

2.4 Server hostnames / SAN plan and fully-qualified URLs

Description: The exact **hostname(s)** and any load-balancer/vanity names that clients use to reach C2M, planned into the certificate **Common Name (CN)** and **Subject Alternative Names (SAN)**. **Why:** TLS validates the certificate against the hostname in the URL; a mismatch (or a missing SAN for the load-balancer name) fails validation even with a valid certificate. **Verify:** The list of DNS names clients use is documented and each is represented in the planned CN/SAN. **Implement:** Collect the DNS names (including any front-end/load-balancer FQDN), and include them in the CSR's SAN extension before requesting the certificate.

2.5 Identity Provider / SSO integration decision

Description: The enterprise SSO mechanism to integrate — **SAML**, **OIDC**, or an access-manager agent — and the IdP-side details (SAML metadata / signing certificate, or OIDC client, or agent configuration) that WebLogic will consume. **Why:** WebLogic's Identity Assertion Provider validates the asserted identity; the OUAF web application must be switched to `CLIENT-CERT` to accept it. Without the IdP side provisioned, SSO cannot be asserted. **Verify:** The identity team can supply the IdP metadata / signing certificate (SAML) or client registration (OIDC) / agent details, and the header/assertion the perimeter will pass is defined. **Implement:** Engage the identity team to register C2M as a relying party / client / protected resource and to provide the metadata/signing certificate and the asserted username claim/header.

2.6 Keystore passphrase custody and secrets handling

Description: A secure custody process for **keystore and private-key passphrases** (a secrets manager or sealed credential store), and agreement that passphrases are **never** committed to source, printed, or placed in the docs/exports. **Why:** Compromise of a private key or its passphrase compromises the server's identity; secrets must be handled out-of-band. The automation in this solution is read-only for state and never prints key material. **Verify:** A vault/credential store is identified; the team agrees passphrases are entered interactively or injected from the vault, not stored in scripts. **Implement:** Provision passphrase storage; document who holds custody and how a passphrase is retrieved for a change window.

2.7 Change control, backups, and a rollback path

Description: A change window and verified **backups** before any SSL/SSO change: the WebLogic **domain config** (`config.xml` and the `security` directory), the OUAF **product/config** (`$SPLEBASE/etc`), and the existing keystores — plus a defined rollback (revert config, restore the previous keystore, or fall back to the prior working state). **Why:** SSL/SSO changes affect every user and integration; a bad change can lock everyone out, so a tested restore is mandatory (see `global-requirements-oracle.md` §5). **Verify:** You have current copies of `config.xml`, the domain `security` folder, `$SPLEBASE/etc`, and each keystore, and a documented restore procedure. **Implement:** Take the backups as the first step of the change; rehearse the restore in a lower environment.

2.8 A non-production pilot server / environment

Description: A single managed server or lower environment where keystores, SSL, and SSO are proven **first**, before the domain is expanded. **Why:** The model is Assess → Configure → **Pilot** → Validate → Expand; a pilot contains blast radius and validates the exact steps against your topology. **Verify:** The pilot server/environment is identified and you can enable an SSL port on it without affecting production traffic. **Implement:** Choose the pilot (e.g. Test, or one managed server), and record it as `Security Target – Pilot – Stage1`.

Checklist

- WebLogic Console + WLST access to the C2M domain confirmed
- OUAF product account, `$SPLEBASE`, `configureEnv`/`initialSetup`, and JDK `keytool` available
- Certificate source and CA path decided; root/intermediate CA files in hand
- Hostname/SAN plan and fully-qualified HTTPS URLs documented

- SSO mechanism chosen; IdP metadata/signing cert or client/agent details available
- Passphrase custody (vault) in place; secrets never in scripts/exports
- Backups taken (config.xml, domain security dir, `$SPLEBASE/etc`, keystores); rollback documented
- Pilot server/environment chosen and recorded as `Security Target – Pilot – Stage1`

Global Requirements — Oracle Utilities C2M / WebLogic Solutions

These requirements apply to **every** solution in the **Oracle** category (Oracle Utilities Customer to Meter and its Oracle WebLogic Server middleware). Verify each before starting a solution's steps.

> ***Vendor & version boundary (applies to every Oracle solution).*** *Oracle Utilities* ***Customer to Meter (C2M)****,* ***Customer Care & Billing (CC&B)****,* ***Meter Data Management (MDM)****, and* ***Oracle WebLogic Server*** *are authored from Oracle's official product documentation and the Oracle Utilities Application Framework (OUAF) install/admin guides.* ***Version-specific details*** *(C2M 25.4 / minimum v2.9, WebLogic 14c) are* ***cited or flagged "verify for your version"*** *— never invented. Some Oracle documentation and patches are entitlement-gated on* ***My Oracle Support (MOS)****; where a step requires a MOS note, patch number, or download, the solution points to it rather than reproducing gated content. All work assumes a* ***non-production environment first*** *and change control for production.*

1. Licensed Oracle Utilities C2M + WebLogic entitlement

- **Description:** A licensed Oracle Utilities **Customer to Meter** (and the underlying OUAF) plus the supported **Oracle WebLogic Server 14c** and database (Oracle Database) at the versions in scope.
- **Why required:** Installation media, patches, and support all flow from the entitlement.
- **How to verify:** Confirm the license and access to **My Oracle Support**; check the certified version matrix for C2M ↔ WebLogic ↔ Database ↔ Java.
- **How to implement if missing:** Obtain licensing/entitlement and MOS access before proceeding.

2. Certified platform stack (OS, Java, Database, WebLogic)

- **Description:** A host OS, JDK, Oracle Database, and WebLogic version combination **certified** for the target C2M release (per Oracle's certification/compatibility matrix).
- **Why required:** C2M/OUAF only supports certified combinations; uncertified stacks cause install and patch failures and lose supportability.
- **How to verify:** Compare the environment against the C2M release's certification matrix (MOS).
- **How to implement if missing:** Align OS/JDK/DB/WebLogic to a certified combination first.

3. Environment topology (Dev / Test / Prod) & access

• **Description:** Defined **Development, Test, and Production** environments (and any QA/UAT), with OS

accounts (e.g. the OUAF/`splunk`-style product user), WebLogic admin credentials, and DB schema access appropriate to the task.

• **Why required:** C2M is deployed and promoted across environments; each solution assumes the target

environment exists and you can reach WebLogic, the file system, and the database.

• **How to verify:** Confirm you can log in to the OS product account, the WebLogic Administration

Console / WLST, and the C2M database schema.

• **How to implement if missing:** Stand up / obtain access to the environment before starting.

4. Administrative access & tooling (WLST, shell, SQL)

• **Description:** Administrative access and the standard toolchain: **WLST (Jython)** for WebLogic,

the OS **shell** (bash/ksh) for OUAF utilities (e.g. `splmanager`, `configureEnv`, `initialSetup`, batch/threadpool scripts), and **SQL*Plus / SQLcl** for the database.

• **Why required:** C2M/WebLogic administration is performed through these interfaces, not a GUI-only

workflow; the solutions' automation is authored as WLST/shell/SQL.

• **How to verify:** Confirm WLST connects to the domain, shell access to the product home, and SQL

connectivity to the schema.

• **How to implement if missing:** Provision the access and install the client tooling.

5. Change control, backup & a non-production pilot

• **Description:** A **change-control** process for production changes, verified **backups** (domain

config, OUAF product/config, database) before disruptive actions (install, patch, upgrade, migration, tuning), and a **non-production environment** to validate on first.

• **Why required:** Every solution follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a

documented rollback; C2M patches/upgrades and WebLogic changes are high-impact.

• **How to verify:** Confirm a backup/restore path and a change window/approval for production.

• **How to implement if missing:** Establish backups and change control before touching production.

6. Collaborating teams & support model

• **Description:** Engagement with the **infrastructure, DBA, network, application-development, and

release/test **teams, and (post go-live) the** hypercare / on-call** support model and monitoring stack (e.g. Dynatrace) the environment uses.

• **Why required:** C2M administration is cross-functional; issues span middleware, database, network,

and application layers, and production support runs on a defined escalation and monitoring model.

• **How to verify:** Confirm the named contacts, escalation path, and monitoring tooling.

• **How to implement if missing:** Identify the teams and the support/monitoring model before go-live.

*The deployment model across every Oracle solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **WLST (Jython), shell, and SQL** scripts — not PowerShell.*

3. Implementation Guide — Oracle C2M SSO, SSL & Certificate/Keystore Configuration

Follow these steps **in order**. Keystores are created with `keytool`; WebLogic SSL and the Identity Assertion Provider are set through the **Administration Console** or **WLST**; OUAF's advertised URL/protocol is set through `configureEnv` and materialized by `initialSetup`. Artifact names follow the `<Purpose> – <Scope> – <Stage>` convention from the Overview and are reused verbatim. Do everything on the **pilot** first (`Security Target – Pilot – Stage1`), pass the gate, then expand.

> ***Model:*** *Assess the current state → build keystores → enable SSL/TLS on the domain → align C2M to HTTPS → wire SSO → validate secure sign-in and endpoints on the pilot → expand across the domain. Pattern:* ***Assess*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand.***
>
> ***Order matters.*** *Keystores (trust chain + identity) must exist and be valid* ***before*** *SSL can point at them; SSL/TLS must terminate correctly* ***before*** *SSO is layered on; SSO is proven* ***last*** *because it depends on both. Never enable two-way SSL or* `CLIENT-CERT` *before one-way TLS is confirmed working.*
>
> ***Placeholders are literal names, not fill-ins:*** *where a value is site-specific it is written as a plain token such as* `your-hostname`*,* `your-keystore-dir`*, or* `your-idp-metadata` *— substitute your real value; do not type angle brackets.*

Step 0 — Assess the current state (Assess)

1. Run `OC2MSEC-Discovery.sh` (read-only shell) to capture the baseline `Security Baseline – C2M –

Discovery: it inventories the keystore **files** on disk with keytool -list – aliases, entry types, and certificate **expiry** dates – reads each server's SSL/keystore attributes from the domain config.xml, and shows the OUAF advertised URL/protocol from $SPLEBASE/etc`.

1. Run `OC2MSEC-Validation.py` (read-only WLST) once connected to confirm the live server MBean view of

the **KeyStores** rule (Demo vs Custom), SSL enabled/port, client-cert enforcement, min TLS, and the configured authentication providers.

1. Confirm what must change: demo keystores present, SSL off, HTTP-only URLs, or no SSO asserter.

Step 1 — Create the identity keystore and CSR (Configure)

1. Generate the server key pair into the identity keystore (alias `c2msec-server`), with CN/SAN covering

every client-facing hostname (Requirement 2.4):

```
keytool -genkeypair -alias c2msec-server -keyalg RSA -keysize 2048 \
  -dname "CN=your-hostname,OU=Utilities,O=your-org,C=US" \
  -ext "SAN=dns:your-hostname,dns:your-lb-hostname" \
  -keystore your-keystore-dir/C2MSEC-Identity-Pilot.jks -storetype JKS
```

Enter the keystore and key passphrases at the prompt (Requirement 2.6) — never on the command line.

1. Produce a CSR and submit it to your CA (Requirement 2.3):

```
keytool -certreq -alias c2msec-server -file your-keystore-dir/c2msec-server.csr \
  -keystore your-keystore-dir/C2MSEC-Identity-Pilot.jks
```

1. For a **lower** environment only, a self-signed certificate from Step 1.1 is acceptable and no CSR is needed; production must use a CA-signed certificate.

Step 2 — Build the trust keystore and complete the chain (Configure)

1. Create the trust keystore and import the CA chain — **root first, then intermediate** — with the standard aliases:

```
keytool -importcert -trustcacerts -alias c2msec-rootca -file your-rootCA.pem \
  -keystore your-keystore-dir/C2MSEC-Trust-Pilot.jks -storetype JKS
keytool -importcert -trustcacerts -alias c2msec-intca -file your-intCA.pem \
  -keystore your-keystore-dir/C2MSEC-Trust-Pilot.jks
```

1. Import the **signed server certificate** back into the *identity* keystore under the **same alias** `c2msec-server` (the reply must chain to the CA certs already trusted):

```
keytool -importcert -alias c2msec-server -file your-signed-server.pem \
  -keystore your-keystore-dir/C2MSEC-Identity-Pilot.jks
```

1. Verify both keystores: `keytool -list -v` shows the identity alias as a **PrivateKeyEntry** with a full chain, and the trust store shows the CA **trustedCertEntry** entries.

Step 3 — Point WebLogic at the keystores (Configure)

Do this via WLST (repeatable) or the Console (`Environment` → `Servers` → `your-server` → `Configuration` → `Keystores`). WLST, against the pilot server MBean:

```
edit(); startEdit()
cd('/Servers/your-server')
cmo.setKeyStores('CustomIdentityAndCustomTrust')
cmo.setCustomIdentityKeyStoreFileName('your-keystore-dir/C2MSEC-Identity-Pilot.jks')
cmo.setCustomIdentityKeyStoreType('JKS')
cmo.setCustomIdentityKeyStorePassPhrase('from-your-vault')   # injected, never hard-coded
cmo.setCustomTrustKeyStoreFileName('your-keystore-dir/C2MSEC-Trust-Pilot.jks')
cmo.setCustomTrustKeyStoreType('JKS')
cmo.setCustomTrustKeyStorePassPhrase('from-your-vault')
save(); activate()
```

The passphrase is retrieved from the vault at runtime (Requirement 2.6), never written into a stored script.

Step 4 — Enable SSL/TLS on the domain (Configure)

1. Enable the SSL listen port and bind the private key, on the pilot server's SSL MBean:

```
edit(); startEdit()
cd('/Servers/your-server/SSL/your-server')
cmo.setEnabled(true)
```

```
cmo.setListenPort(7002)                       # your chosen SSL port
cmo.setServerPrivateKeyAlias('c2msec-server')
cmo.setServerPrivateKeyPassPhrase('from-your-vault')
cmo.setClientCertificateEnforced(false)       # one-way TLS first; two-way is Step 8
cmo.setMinimumTLSProtocolVersion('TLSv1.2')   # verify attribute name for your version
save(); activate()
```

1. Restart SSL (Console **Control** → **Restart SSL**, or restart the managed server) and confirm the SSL port is listening. Do **not** enable client-cert enforcement yet.

1. Retire the demo keystores from any server still using them — demo certs are never valid for production trust.

Step 5 — Align C2M/OUAF to HTTPS (Configure)

1. As the product account, run `configureEnv` and set the web application server URL/protocol and port to the **HTTPS** endpoint (the SSL port from Step 4), so OUAF advertises `https` in links, callbacks, and inbound web-service URLs.

1. Run `initialSetup` to regenerate `web.xml`, `weblogic.xml`, and `spl.properties` from the templates with the new values. (Editing the generated files directly is overwritten on the next `initialSetup` — always change through `configureEnv`/templates; see Requirement 2.2.)

1. Redeploy/restart the OUAF applications as your install requires, and confirm C2M loads over `https` with the new certificate (no browser warning when CA-signed).

Step 6 — Configure the SSO Identity Assertion Provider (Configure)

1. In the WebLogic security realm, add the Identity Assertion Provider that matches your SSO mechanism (Requirement 2.5) and name it `C2MSEC-SSO-Asserter` (`Security Realms` → `myrealm` → `Providers` → `Authentication` → `New`). For SAML, configure the SAML2 Identity Asserter and import the IdP **signing certificate** into the **trust** keystore (alias e.g. `c2msec-idpsign`); for an access-manager agent, configure the corresponding asserter for the asserted header/token.

1. Set the provider's active token/header type to the one the perimeter passes, and order the providers so the asserter runs ahead of the default authenticator. Restart the Administration Server if required for the new provider to take effect.

Step 7 — Switch OUAF to accept the asserted identity (Configure)

1. Through `configureEnv`, set the OUAF web authentication method to `CLIENT-CERT` (so the application accepts the WebLogic-asserted user instead of showing its own form login), then run `initialSetup` to regenerate `web.xml`/`weblogic.xml` with that auth-method.

1. Confirm the asserted username maps to a valid **C2M user** (the username claim/header the IdP sends must resolve to a C2M user record). Redeploy/restart the OUAF applications.

Step 8 — Pilot two-way SSL where required (Pilot)

1. If integrations require **mutual TLS**, first import each trusted client/issuer CA into

`C2MSEC-Trust-Pilot.jks`, then set enforcement on the pilot server:

```
cd('/Servers/your-server/SSL/your-server')
cmo.setClientCertificateEnforced(true)        # 'need'; use two-way SSL host verification as required
save(); activate()
```

1. Restart SSL and confirm a caller **with** a trusted client cert connects and one **without** is

correctly rejected. Enforce two-way SSL only where it is actually required — it breaks callers that lack a trusted client certificate.

Step 9 — Validate the pilot (Validate)

1. Run `OC2MSEC-Validation.py` (read-only) to produce `Security Validation –

C2M – Validation: SSL port up, min TLS/cipher policy correct, identity chain valid and not near expiry, trust contains the expected CAs, asserter present, and OUAF on CLIENT-CERT`.

1. Run `OC2MSEC-Report.py` to produce `Security Coverage – C2M – Report` — a per-server coverage sheet

(Custom vs Demo, SSL on/off, TLS floor, cert expiry, SSO wired).

1. Complete the **Testing Guide** and pass the gate: SSO sign-in works, HTTPS endpoints are trusted,

keystores validate, and no user or integration is broken.

Step 10 — Expand across the domain (Expand)

1. Run `OC2MSEC-ScopeHelper.py` to enumerate the remaining in-scope servers as `Security Target – Domain

– Stage2` and to stage the same keystore/SSL/SSO settings per server (each server presents its own identity certificate; trust and asserter are consistent).

1. Apply Steps 3–7 to each domain server (reusing the same aliases and provider name), re-run

`OC2MSEC-Validation.*` and `OC2MSEC-Report.py`, and confirm domain-wide coverage.

1. Record certificate **expiry** dates and schedule rotation (see Best Practices) so trust never lapses.

Order of operations

`Assess → Identity keystore + CSR → Trust chain → Point WebLogic at keystores → Enable one-way SSL → Align C2M to HTTPS → SSO asserter → OUAF CLIENT-CERT → (optional) two-way SSL → Validate pilot → Expand domain`

5. Testing Guide — Oracle C2M SSO, SSL & Certificate/Keystore Configuration

Run after each configuration stage. Every test is **read-only or non-destructive**; the pilot is the gate before the domain is expanded. Use the read-only scripts for state and standard client tools (browser, `openssl s_client`, `keytool -list`) to confirm from the outside.

Test 1 — Baseline captured

Do: Run `OC2MSEC-Discovery.sh` (on-disk keystores + config.xml + OUAF URL) and `OC2MSEC-Validation.py` (live server MBeans). **Expected:** `Security Baseline – C2M – Discovery` lists every server's KeyStores rule, identity/trust files, private-key alias, SSL port/state, client-cert enforcement, TLS floor, and auth providers; the on-disk keystore inventory shows aliases and expiry. No demo keystore remains flagged for a production server after Step 4.

Test 2 — Keystores are valid and chained

Do: `keytool -list -v` on the identity and trust keystores (or read the `OC2MSEC-Discovery.sh` output). **Expected:** Identity alias `c2msec-server` is a **PrivateKeyEntry** with a full chain to the CA; trust holds `c2msec-rootca` (and `c2msec-intca`) as **trustedCertEntry**; no entry is expired or within the rotation threshold.

Test 3 — One-way SSL/TLS terminates correctly

Do: From a client, `openssl s_client -connect your-hostname:7002` (or load the C2M HTTPS URL in a browser). **Expected:** The server presents `c2msec-server`, the chain validates against the enterprise CA (no warning when CA-signed), and the negotiated protocol is **TLS 1.2+**. HTTP-only access is gone or redirected.

Test 4 — Minimum TLS and cipher policy enforced

Do: Attempt a TLS 1.0/1.1 handshake (`openssl s_client -tls1_1 ...`) and inspect negotiated ciphers. **Expected:** Legacy protocols are refused; only the approved TLS 1.2+ ciphers negotiate — matching `Security Validation – C2M – Validation`.

Test 5 — C2M advertises HTTPS

Do: Open C2M and inspect generated links/callbacks; check `$SPLEBASE/etc` values via `OC2MSEC-Discovery.sh`. **Expected:** OUAF advertises the `https` endpoint (from `configureEnv`/`initialSetup`); inbound web-service URLs and in-app links use `https`, not `http`.

Test 6 — SSO sign-in works

Do: Access C2M through the enterprise SSO path with a test user. **Expected:** The IdP authenticates the user, WebLogic's `C2MSEC-SSO-Asserter` asserts identity, OUAF (on `CLIENT-CERT`) accepts it and maps to the correct C2M user — **no** separate C2M form-login prompt.

Test 7 — SSO negative path is safe

Do: Attempt access without a valid SSO assertion / with an untrusted assertion. **Expected:** Access is denied cleanly (no fallback that bypasses SSO); the asserter rejects an assertion whose signing certificate

is not in the trust keystore.

Test 8 — Two-way SSL enforcement (only where configured)

Do: On a server with `ClientCertificateEnforced=true`, connect once **with** a trusted client cert and once **without**. **Expected:** The trusted client connects; the untrusted/absent-cert client is rejected — mutual TLS is actually enforced, and only on the intended servers.

Test 9 — Validation and coverage are clean

Do: Run `OC2MSEC-Validation.py` and `OC2MSEC-Report.py`. **Expected:** `Security Validation – C2M – Validation` shows all checks pass for the pilot; `Security Coverage – C2M – Report` shows the pilot server Custom+SSL+SSO with a valid, non-expiring certificate.

Test 10 — Domain expansion is consistent

Do: After Step 10, re-run `OC2MSEC-Report.py` across `Security Target – Domain – Stage2`. **Expected:** Every in-scope server reports Custom Identity/Trust, SSL on, TLS 1.2+ floor, SSO asserter present, OUAF on `CLIENT-CERT`, and each server presents its own valid certificate.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–9 pass on `Security Target – Pilot – Stage1`: keystores valid, one-way TLS 1.2+ terminating, C2M on HTTPS, SSO sign-in working and negative path safe, (if used) two-way SSL enforced — with zero broken users/integrations
Domain pass	Test 10 passes for `Security Target – Domain – Stage2`; every server reports Custom keystores, SSL on, SSO wired, and valid non-expiring certificates before rotation is scheduled and change control is closed

6. Troubleshooting Guide — Oracle C2M SSO, SSL & Certificate/Keystore Configuration

Use `OC2MSEC-Discovery.sh` (on-disk keystores, `config.xml`, OUAF URL) and `OC2MSEC-Validation.py` (live server MBeans) for current state, re-run `OC2MSEC-Validation.py` to confirm a fix, and `OC2MSEC-Remediation.py` (**dry-run first**) for safe changes. WebLogic SSL diagnostics come from the managed server log and `-Dweblogic.ssl.Verbose=true` / `-Dssl.debug=true` where deeper handshake tracing is needed. None of the tooling prints private keys or passphrases.

A. Keystore / trust errors

Symptom	Cause	Remediation
Server fails to start SSL: private key not found	The SSL private-key alias does not match the identity keystore entry	Set `ServerPrivateKeyAlias` to `c2msec-server` (the alias used in Step 1); confirm with `keytool -list`
`unable to find valid certification path` on server-to-server calls	The peer/IdP CA is not in the trust keystore	Import the missing root/intermediate CA into `C2MSEC-Trust-<env>.jks` (`c2msec-rootca` / `c2msec-intca`) and restart SSL
Identity alias shows as `trustedCertEntry`, not `PrivateKeyEntry`	The signed cert was imported under a new alias instead of the original key alias	Re-import the CA reply under the **same** alias `c2msec-server` so it binds to the private key
Keystore load fails / bad password	Wrong keystore or key passphrase	Retrieve the correct passphrase from the vault (Requirement 2.6); never hard-code it

B. SSL/TLS won't terminate or is untrusted

Symptom	Cause	Remediation
Browser warning "not trusted"	Self-signed cert, or client missing the enterprise CA	Use a CA-signed cert in production; ensure the client trusts the issuing CA
Hostname mismatch error	Certificate CN/SAN omits the URL's hostname (often the load-balancer FQDN)	Reissue with the hostname in the **SAN** (Requirement 2.4)
Handshake fails after WebLogic/JDK patch	Patch changed default protocols/ciphers	Re-set the TLS floor to 1.2+ and the approved cipher list; re-validate with `OC2MSEC-Validation.py`
SSL port not listening	SSL not enabled or server not restarted	Confirm `Enabled=true` and the listen port; restart SSL (Console **Restart SSL**) or the server

C. C2M still using HTTP

- Links/callbacks show `http` → the OUAF advertised URL/protocol was not updated, or the change was

made only in the generated files. Set the HTTPS endpoint via `configureEnv`, run `initialSetup` to regenerate `web.xml`/`weblogic.xml`/`spl.properties`, and redeploy/restart (Requirement 2.2).

D. SSO does not sign the user in

Symptom	Cause	Remediation
C2M shows its own form login instead of SSO	OUAF not on `CLIENT-CERT`, or the auth-method change was overwritten by `initialSetup`	Set `CLIENT-CERT` via `configureEnv`, re-run `initialSetup`, redeploy
Assertion rejected / user not asserted	`C2MSEC-SSO-Asserter` missing, mis-ordered, or the IdP signing certificate not trusted	Confirm the asserter exists and runs ahead of the default authenticator; import the IdP signing cert into trust
Asserted but "user not found"	The IdP username claim/header does not map to a C2M user	Align the asserted username to a valid C2M user record; correct the claim/header mapping
SSO works but bypass is possible	A fallback path skips the asserter	Close the fallback so the negative-path test denies access (Testing Test 7)

E. Two-way SSL problems

• Legitimate caller rejected → its client certificate's issuer CA is not in the trust keystore, or

enforcement was turned on before trust was ready. Import the client/issuer CA into trust, then confirm `ClientCertificateEnforced=true` only on the intended servers. If two-way SSL was enabled prematurely, set enforcement back to `false`, restore one-way TLS, and re-stage.

F. Certificate expiry / rotation

• TLS suddenly fails for everyone on a specific date → the server or a CA certificate **expired**.

Check expiry with `OC2MSEC-Discovery.sh`; import the renewed certificate under `c2msec-server` (or the renewed CA into trust), restart SSL, and bring the rotation schedule forward (Best Practices).

G. Change made no difference / was lost

• A `web.xml`/`weblogic.xml` edit disappeared → it was made in a **generated** file and overwritten by

`initialSetup`. Always change through `configureEnv`/templates. A WLST change that didn't apply → the edit session was not activated; re-run inside `startEdit()` … `save(); activate()`.

H. Diagnostics quick reference

```
keytool -list -v -keystore your-keystore-dir/C2MSEC-Identity-Pilot.jks   # aliases, chain, expiry
keytool -list -v -keystore your-keystore-dir/C2MSEC-Trust-Pilot.jks      # trusted CA entries
openssl s_client -connect your-hostname:7002 -servername your-hostname   # protocol, chain, cert
java weblogic.WLST OC2MSEC-Validation.py                                 # read-only full check
```

(Never pass keystore passphrases on the command line; enter them at the prompt or inject from the vault.)

I. When to escalate

If keystores validate, the CA chain and IdP signing certificate are trusted, one-way TLS terminates on a modern floor, OUAF is on `CLIENT-CERT`, and SSO still fails, capture the managed-server SSL/handshake log, the asserter configuration, and the (non-secret) `OC2MSEC-Discovery.sh` output, and open a **My Oracle Support** case for the C2M/OUAF and WebLogic versions in scope — some SSO/IdP interoperability notes are MOS-gated.

7. Best Practices — Oracle C2M SSO, SSL & Certificate/Keystore Configuration

- **Separate identity and trust keystores, always.** Keep the private key and server certificate in the **identity** keystore (`C2MSEC-Identity-<env>.jks`) and CA certificates in the **trust** keystore (`C2MSEC-Trust-<env>.jks`). Mixing them makes rotation and least-privilege custody harder.
- **Never run production on demo keystores.** `DemoIdentity.jks` / `DemoTrust.jks` are for illustration only; retire them as part of Step 4 and confirm via `OC2MSEC-Discovery.sh` that no production server still references them.
- **CA-signed in production; self-signed only in lower environments.** A self-signed cert produces warnings and breaks server-to-server trust — acceptable in Dev/Test, never in Prod.
- **Plan CN and SAN for every client-facing name.** Include the load-balancer/vanity FQDN in the SAN before requesting the certificate; a missing SAN fails hostname validation even with a valid cert.
- **Configure in the safe order.** Build keystores → one-way TLS → align C2M to HTTPS → SSO → (only if required) two-way SSL. Never enable `CLIENT-CERT` or client-cert enforcement before plain one-way TLS is confirmed working.
- **Enforce a modern TLS floor.** Set minimum TLS to **1.2+** and a curated cipher list; refuse TLS 1.0/1.1. Re-check after WebLogic/JDK patching, which can shift defaults.
- **Own the certificate lifecycle — rotation is scheduled, not hoped for.** Record every certificate's **expiry** from `OC2MSEC-Discovery.sh`, alert well ahead (e.g. 60/30 days), and rotate on a calendar. Rotate by generating a new key/CSR, importing the new signed cert under the **same alias** `c2msec-server` (or a versioned alias with a cutover), and restarting SSL — in a change window with a backup of the previous keystore for rollback.
- **Refresh trust before it lapses.** Import renewed root/intermediate CA certs into trust ahead of the old ones expiring; keep IdP signing certificates current in trust so SSO assertions keep validating.
- **Protect passphrases and private keys.** Keep keystore/key passphrases in a vault, inject them at change time, and restrict OS permissions on keystore files to the product account. Never place passphrases or keys in scripts, docs, exports, or `config.xml` in clear form.
- **Persist changes through OUAF templates.** Set the HTTPS URL/protocol and the `CLIENT-CERT` auth-method via `configureEnv`, then `initialSetup`; changes made only to generated `web.xml`/`weblogic.xml` are lost on the next regeneration.

• **Keep the SSO fallback closed.** Ensure there is no path that silently bypasses the asserter; the negative-path test (untrusted/absent assertion) must deny access.

• **Pilot, then expand — with rollback ready.** Prove the whole chain on one server/environment, keep backups of `config.xml`, the domain `security` directory, `$SPLEBASE/etc`, and each keystore, and only then apply the same named artifacts across the domain.

• **Make security state auditable.** Run the read-only `Discovery` → `Validation` → `Report` chain on a schedule so drift (a demo keystore reappearing, SSL turned off, a cert nearing expiry, TLS floor lowered) is caught before it becomes an outage or an audit finding.

8. Deliverables Summary — Oracle C2M SSO, SSL & Certificate/Keystore Configuration

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The C2M security model — SSO, SSL/TLS termination, and the identity/trust keystore chain on WebLogic — with the naming convention and limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Oracle)	Category-wide prerequisites (entitlement, certified stack, environments, tooling, change control, teams), published once
3. Implementation Guide	Exact, ordered `keytool` + WLST/Console + `configureEnv`/`initialSetup` steps (assess → keystores → one-way SSL → HTTPS for C2M → SSO asserter → OUAF `CLIENT-CERT` → two-way SSL → validate → expand)
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Certificate lifecycle, key rotation, and safe-order recommendations

Automation scripts (WLST / shell / SQL — never PowerShell)

Script	What it does
`OC2MSEC-Discovery.sh`	**Read-only** shell discovery: `keytool -list` inventory of keystore files (aliases, entry types, certificate expiry), each server's SSL/keystore attributes read from the domain `config.xml`, and the OUAF advertised URL/protocol from `$SPLEBASE/etc`. Writes `Security Baseline – C2M – Discovery`. Never prints private keys or passphrases
`OC2MSEC-Validation.py`	**Read-only** WLST checks against the live server MBeans: keystore rule Custom (not Demo), SSL enabled on the expected port, TLS floor correct, identity chain valid and not near expiry, trust holds the expected CAs, `C2MSEC-SSO-Asserter` present and ordered, OUAF on `CLIENT-CERT`. Writes `Security Validation – C2M – Validation`
`OC2MSEC-Report.py`	**Read-only** WLST per-server coverage report (Custom vs Demo, SSL on/off, TLS floor, certificate expiry, SSO wired). Writes `Security Coverage – C2M – Report`
`OC2MSEC-Remediation.py`	Safe WLST remediation (point servers at Custom Identity/Trust, enable SSL, set TLS floor, set client-cert enforcement, add/order the asserter). **DRY-RUN by default**; `-execute` to apply. Reads passphrases from the vault/prompt at runtime; never prints or stores secrets
`OC2MSEC-ScopeHelper.py`	Enumerates in-scope servers and stages the pilot (`Security Target – Pilot – Stage1`) vs domain (`Security Target – Domain – Stage2`) target sets so keystore/SSL/SSO settings are applied consistently per server

What the buyer receives

- This implementation guide as a branded **Word** document and **PDF**.
- All **automation scripts** (WLST/Jython, shell) in a single ZIP.
- A repeatable process following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

C2M secured end to end — **SSO** working through the enterprise IdP, **SSL/TLS** valid on the WebLogic domain and C2M, and the **identity/trust keystores** correct, trusted, and on a rotation schedule — proven on a pilot server before the domain is expanded, with secrets never exposed by the tooling.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OC2MSEC-Discovery.sh

Purpose: Capture the security baseline from the file system and the domain config, without connecting a live edit session. It lists the identity and trust keystore contents with keytool (aliases, entry types, certificate validity/expiry), reads each server's SSL and keystore attributes from the domain config.xml, and reports the OUAF advertised URL/protocol from the product config. Set the inputs as environment variables (no angle brackets): OC2MSEC_IDENTITY, OC2MSEC_TRUST, OC2MSEC_DOMAIN_HOME, SPLEBASE. HOW TO RUN export OC2MSEC_IDENTITY=your-keystore-dir/C2MSEC-Identity-Pilot.jks export OC2MSEC_TRUST=your-keystore-dir/C2MSEC-Trust-Pilot.jks export OC2MSEC_DOMAIN_HOME=your-domain-home export SPLEBASE=your-product-home sh OC2MSEC-Discovery.sh EXPECTED RESULTS A written baseline (Security-Baseline-C2M-Discovery.txt) plus on-screen output listing keystore aliases and certificate expiry, the config.xml SSL/keystore lines per server, and the OUAF https/http endpoint. PASS when identity shows a PrivateKeyEntry that is not expired and trust holds the expected CA entries; flag when a Demo keystore or an expired/near-expiry cert appears. NOTES White-label — the repeatable way to inventory C2M security state before changing it. Read-only; keytool -list never emits private keys, and this script never echoes passphrases (it prompts keytool interactively or lists without a store password where the JDK allows).

How to run: export OC2MSEC_IDENTITY=your-keystore-dir/C2MSEC-Identity-Pilot.jks export OC2MSEC_TRUST=your-keystore-dir/C2MSEC-Trust-Pilot.jks export OC2MSEC_DOMAIN_HOME=your-domain-home export SPLEBASE=your-product-home sh OC2MSEC-Discovery.sh EXPECTED RESULTS A written baseline (Security-Baseline-C2M-Discovery.txt) plus on-screen output listing keystore aliases and certificate expiry, the config.xml SSL/keystore lines per server, and the OUAF https/http endpoint. PASS when identity shows a PrivateKeyEntry that is not expired and trust holds the expected CA entries; flag when a Demo keystore or an expired/near-expiry cert appears. NOTES White-label — the repeatable way to inventory C2M security state before changing it. Read-only; keytool -list never emits private keys, and this script never echoes passphrases (it prompts keytool interactively or lists without a store password where the JDK allows).

OC2MSEC-Remediation.py

Purpose: Bring a target server to the intended secure posture: point it at the Custom Identity and Custom Trust keystores, set the server private-key alias, enable SSL on the intended port, set the minimum TLS floor, and set client-certificate enforcement. By default it only PRINTS the changes it would make (dry run) and changes nothing; pass -execute to apply inside a proper edit session. Keystore and key passphrases are read at runtime from the vault helper or an interactive prompt, never from a stored value. Inputs are given as environment variables without angle brackets: OC2MSEC_SERVER, OC2MSEC_IDENTITY, OC2MSEC_TRUST, OC2MSEC_ALIAS, OC2MSEC_SSL_PORT, OC2MSEC_TLS_FLOOR. HOW TO RUN java weblogic.WLST OC2MSEC-Remediation.py (dry run — prints intended changes only) java weblogic.WLST OC2MSEC-Remediation.py -execute (applies the changes) Set OC2MSEC_SERVER=your-server, OC2MSEC_IDENTITY=your-keystore-dir/C2MSEC-Identity-Pilot.jks, etc. EXPECTED RESULTS Dry run: a numbered list of the exact attribute settings that WOULD be applied, and the message "DRY RUN - no changes made". With -execute: the same settings applied, then save()/activate() and "Applied and activated". Re-run OC2MSEC-Validation.py to confirm PASS. NOTES

White-label — the repeatable, safe way to remediate C2M security. DRY-RUN by default; never prints private keys or passphrases (they are prompted/injected and referenced only as SET at apply time).

How to run: java weblogic.WLST OC2MSEC-Remediation.py (dry run — prints intended changes only) java weblogic.WLST OC2MSEC-Remediation.py -execute (applies the changes) Set OC2MSEC_SERVER=your-server, OC2MSEC_IDENTITY=your-keystore-dir/C2MSEC-Identity-Pilot.jks, etc. EXPECTED RESULTS Dry run: a numbered list of the exact attribute settings that WOULD be applied, and the message "DRY RUN - no changes made". With -execute: the same settings applied, then save()/activate() and "Applied and activated". Re-run OC2MSEC-Validation.py to confirm PASS. NOTES White-label — the repeatable, safe way to remediate C2M security. DRY-RUN by default; never prints private keys or passphrases (they are prompted/injected and referenced only as SET at apply time).

OC2MSEC-Report.py

Purpose: Produce a single per-server coverage sheet for stakeholders and audit: keystore rule (Custom vs Demo), SSL on/off and port, minimum TLS floor, client-certificate enforcement, server private-key alias, and whether the SSO asserter is wired. It rolls the per-server rows into a coverage summary (how many servers are fully secured vs partial) so domain expansion progress is visible at a glance. Read-only. Writes the report named Security Coverage - C2M - Report. Uses your-admin-url / your-admin-user without angle brackets. HOW TO RUN java weblogic.WLST OC2MSEC-Report.py Connect when prompted (or pre-set WLS_ADMIN_URL / WLS_ADMIN_USER). EXPECTED RESULTS A formatted coverage table on screen and in Security Coverage - C2M - Report, ending with a summary line (e.g. "Secured 3/5: Custom+SSL+SSO"). Read-only: no edit session, nothing changes. NOTES White-label — the repeatable way to report C2M security coverage during pilot and expansion. Read-only; never prints private keys or passphrases.

How to run: java weblogic.WLST OC2MSEC-Report.py Connect when prompted (or pre-set WLS_ADMIN_URL / WLS_ADMIN_USER). EXPECTED RESULTS A formatted coverage table on screen and in Security Coverage - C2M - Report, ending with a summary line (e.g. "Secured 3/5: Custom+SSL+SSO"). Read-only: no edit session, nothing changes. NOTES White-label — the repeatable way to report C2M security coverage during pilot and expansion. Read-only; never prints private keys or passphrases.

OC2MSEC-ScopeHelper.py

Purpose: Enumerate the WebLogic servers in the domain and split them into the pilot scope (Security Target - Pilot - Stage1, typically one managed server or the lower environment) and the domain scope (Security Target - Domain - Stage2, the remaining in-scope servers), so keystore, SSL, and SSO settings are applied consistently and in the right order. It emits, for each scoped server, the exact OC2MSEC_SERVER value and a ready-to-run OC2MSEC-Remediation.py dry-run command, so the operator drives expansion server by server. Read-only — it selects and prints scope; it changes nothing. Set the pilot server with OC2MSEC_PILOT (no angle brackets). HOW TO RUN java weblogic.WLST OC2MSEC-ScopeHelper.py Optionally: export OC2MSEC_PILOT=your-pilot-server (else the first server is treated as pilot). EXPECTED RESULTS Two lists on screen and in Security-Target-Scope.txt: Stage1 (pilot) and Stage2 (domain), each line showing the server name and the dry-run remediation command to run for it. Read-only: no edit session; nothing is changed. NOTES White-label — the repeatable way to sequence pilot-then-domain rollout. Read-only; never prints private keys or passphrases.

How to run: java weblogic.WLST OC2MSEC-ScopeHelper.py Optionally: export OC2MSEC_PILOT=your-pilot-server (else the first server is treated as pilot). EXPECTED RESULTS Two

lists on screen and in Security-Target-Scope.txt: Stage1 (pilot) and Stage2 (domain), each line showing the server name and the dry-run remediation command to run for it. Read-only: no edit session; nothing is changed. NOTES White-label — the repeatable way to sequence pilot-then-domain rollout. Read-only; never prints private keys or passphrases.

OC2MSEC-Validation.py

Purpose: Confirm the live server MBean view is correct after configuration: keystore rule is CustomIdentityAndCustomTrust (not Demo), SSL is enabled on the expected port, the minimum TLS protocol is at or above the floor, the server private-key alias is set, client-certificate enforcement matches intent, and the SSO Identity Assertion Provider (default name C2MSEC-SSO-Asserter) is present in the authentication provider list. Read-only. Writes the result named Security Validation - C2M - Validation. Uses your-admin-url / your-admin-user without angle brackets, and an expected TLS floor of TLSv1.2 by default. HOW TO RUN java weblogic.WLST OC2MSEC-Validation.py Connect when prompted (or pre-set WLS_ADMIN_URL / WLS_ADMIN_USER). Optional expectations: OC2MSEC_TLS_FLOOR (default TLSv1.2), OC2MSEC_ASSERTER (default C2MSEC-SSO-Asserter). EXPECTED RESULTS A PASS/FAIL line per server and an overall result in Security Validation - C2M - Validation: PASS when the server shows Custom keystores, SSL enabled, TLS floor met, private-key alias set, and the asserter present; FAIL with the specific reason otherwise. Read-only: no edit session. NOTES White-label — the repeatable way to prove C2M security before expanding the domain. Read-only; never prints private keys or passphrases (passphrase attributes are reported only as SET/NOT SET).

How to run: java weblogic.WLST OC2MSEC-Validation.py Connect when prompted (or pre-set WLS_ADMIN_URL / WLS_ADMIN_USER). Optional expectations: OC2MSEC_TLS_FLOOR (default TLSv1.2), OC2MSEC_ASSERTER (default C2MSEC-SSO-Asserter). EXPECTED RESULTS A PASS/FAIL line per server and an overall result in Security Validation - C2M - Validation: PASS when the server shows Custom keystores, SSL enabled, TLS floor met, private-key alias set, and the asserter present; FAIL with the specific reason otherwise. Read-only: no edit session. NOTES White-label — the repeatable way to prove C2M security before expanding the domain. Read-only; never prints private keys or passphrases (passphrase attributes are reported only as SET/NOT SET).